

Assignment: 02

CLO 2: Apply computer vision techniques for solving practical problems.

7th , Apr, 2026

Submitted By:

Shafi Azam (SP24-BAI-047)

Muhammad Hamza Habib (SP24-BAI-034)

Submitted To: Dr. Tahir Mustafa Madni

Subject: CV



Department of Computer Science

Comsats University Islamabad

Islamabad Campus

Scattered Data Interpolation

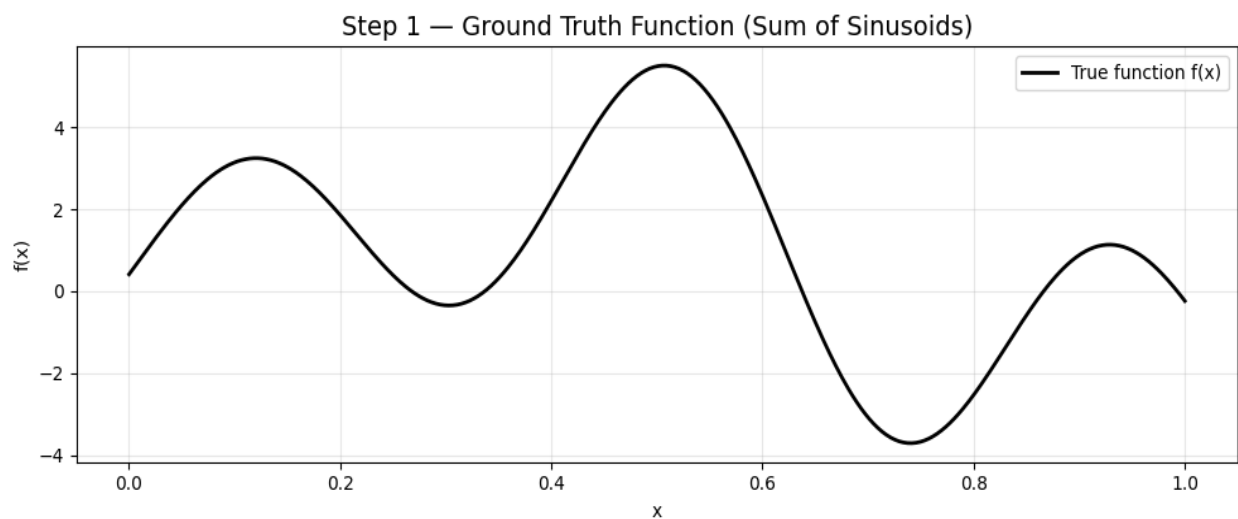
```
# =====  
# Assignment 02: Scattered Data Interpolation  
# CLO 2: Apply computer vision techniques for practical problems  
# =====  
  
import numpy as np  
import matplotlib.pyplot as plt  
from matplotlib import gridspec  
from mpl_toolkits.mplot3d import Axes3D  
  
np.random.seed(42)  
print("Libraries loaded successfully.")  
  
Libraries loaded successfully.
```

Step 1: Generate the Ground-Truth Function

```
# =====  
# STEP 1: Generate a smoothly varying 1-D function  
#         by summing sinusoids of random amplitudes & frequencies  
# =====  
  
def generate_true_function(x, n_components=5, seed=42):  
    """  
    Sum of n_components sinusoids with random amplitudes and frequencies.  
    This is our 'ground truth' function f(x).  
    """  
    rng = np.random.RandomState(seed)  
    amplitudes = rng.uniform(0.5, 2.0, n_components)  
    frequencies = rng.uniform(0.5, 3.0, n_components)  
    phases = rng.uniform(0, 2*np.pi, n_components)  
  
    y = np.zeros_like(x, dtype=float)  
    for A, f, phi in zip(amplitudes, frequencies, phases):  
        y += A * np.sin(2 * np.pi * f * x + phi)  
    return y  
  
# Dense grid for "ground truth"  
x_true = np.linspace(0, 1, 500)  
y_true = generate_true_function(x_true)
```

```
plt.figure(figsize=(10, 4))
plt.plot(x_true, y_true, 'k-', linewidth=2, label='True function f(x)')
plt.title('Step 1 – Ground Truth Function (Sum of Sinusoids)', fontsize=14)
plt.xlabel('x')
plt.ylabel('f(x)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

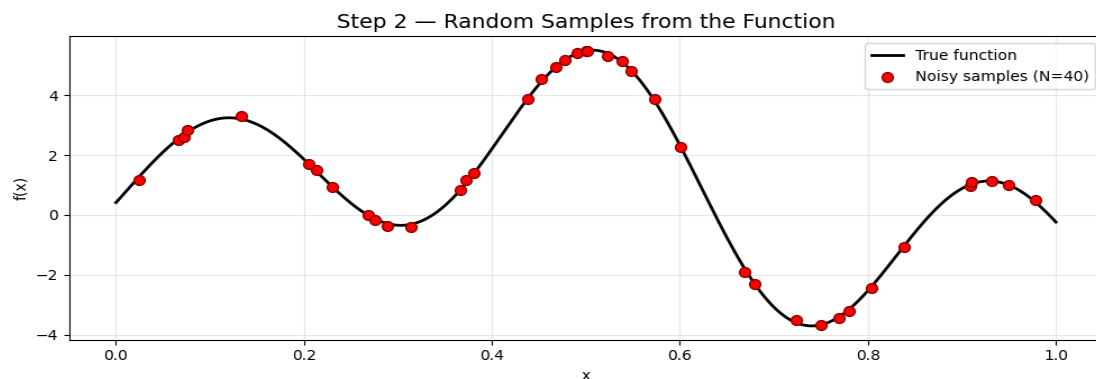
print(f"True function range: [{y_true.min():.3f}, {y_true.max():.3f}]")
```



```
True function range: [-3.711, 5.513]
```

Step 2: Sample at Random Locations

```
# =====  
# STEP 2: Sample the function at N random locations  
# =====  
  
N_SAMPLES = 40 # number of random samples  
  
rng = np.random.RandomState(7)  
x_samples = np.sort(rng.uniform(0, 1, N_SAMPLES)) # sorted for clean plots  
y_samples = generate_true_function(x_samples)  
  
# Add a tiny amount of noise to simulate real measurements  
noise_std = 0.05  
y_noisy = y_samples + rng.normal(0, noise_std, N_SAMPLES)  
  
plt.figure(figsize=(10, 4))  
plt.plot(x_true, y_true, 'k-', linewidth=2, label='True function')  
plt.scatter(x_samples, y_noisy, color='red', zorder=5, s=50,  
           edgecolors='darkred', label=f'Noisy samples (N={N_SAMPLES})')  
plt.title('Step 2 – Random Samples from the Function', fontsize=14)  
plt.xlabel('x')  
plt.ylabel('f(x)')  
plt.legend()  
plt.grid(True, alpha=0.3)  
plt.tight_layout()  
plt.show()  
  
print(f"Sampled {N_SAMPLES} points. Noise std = {noise_std}")
```



Sampled 40 points. Noise std = 0.05

Step 3a: RBF Interpolation (from scratch with NumPy)

```
# =====
# STEP 3: Scattered Data Interpolation Techniques
#
# Technique 1 – Radial Basis Function (RBF) Interpolation
#  $f(x) = \sum_i w_i * \phi(||x - x_i||)$ 
#  $\phi(r) = \exp(-r^2 / (2 * \sigma^2))$  [Gaussian RBF]
# =====

def rbf_fit(x_train, y_train, sigma):
    """
    Fit RBF interpolant.
    Builds NxN kernel matrix Phi and solves Phi @ w = y for weights w.

    Parameters
    -----
    x_train : (N,) array of sample locations
    y_train : (N,) array of sample values
    sigma   : RBF width (bandwidth)

    Returns
    -----
    w       : (N,) weight vector
    """
    # Pairwise distance matrix  $||x_i - x_j||$ 
    diff = x_train[:, None] - x_train[None, :] # (N, N)
    Phi = np.exp(-diff**2 / (2 * sigma**2)) # (N, N) kernel matrix

    # Solve for weights: add small regularisation for numerical stability
    lam = 1e-6
    w = np.linalg.solve(Phi + lam * np.eye(len(x_train)), y_train)
    return w

def rbf_predict(x_new, x_train, w, sigma):
    """
    Predict at new locations using fitted RBF weights.
    """
    diff = x_new[:, None] - x_train[None, :] # (M, N)
    Phi = np.exp(-diff**2 / (2 * sigma**2)) # (M, N)
    return Phi @ w # (M,)

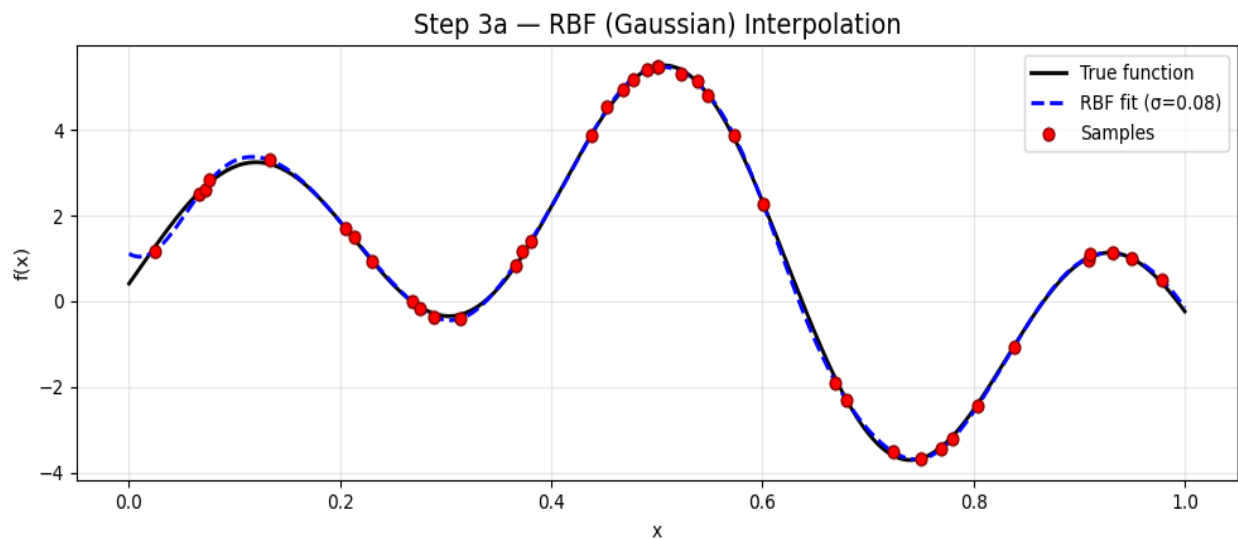
# --- Fit with a chosen sigma ---
sigma_rbf = 0.08
w_rbf = rbf_fit(x_samples, y_noisy, sigma=sigma_rbf)
```

```

y_rbf = rbf_predict(x_true, x_samples, w_rbf, sigma=sigma_rbf)

plt.figure(figsize=(10, 4))
plt.plot(x_true, y_true, 'k-', linewidth=2, label='True function')
plt.plot(x_true, y_rbf, 'b--', linewidth=2, label=f'RBF fit ( $\sigma=\{sigma\_rbf\}$ )')
plt.scatter(x_samples, y_noisy, color='red', zorder=5, s=40,
            edgecolors='darkred', label='Samples')
plt.title('Step 3a – RBF (Gaussian) Interpolation', fontsize=14)
plt.xlabel('x'); plt.ylabel('f(x)')
plt.legend(); plt.grid(True, alpha=0.3)
plt.tight_layout(); plt.show()

```



Step 3b: Polynomial (Least-Squares) Regression

```

# =====
# Technique 2 – Polynomial Least-Squares Regression
#   Build Vandermonde matrix V and solve V @ c = y in least-squares sense
# =====

def poly_fit(x_train, y_train, degree):
    """
    Fit polynomial of given degree using the normal equations (NumPy lstsq).
    Returns coefficient vector c such that  $\sum_k c[k] * x^k \approx y$ .
    """
    # Vandermonde matrix (N, degree+1)
    V = np.vander(x_train, N=degree + 1, increasing=True)
    # Least-squares solution
    c, _, _, _ = np.linalg.lstsq(V, y_train, rcond=None)

```

```

    return c

def poly_predict(x_new, c):
    """Evaluate polynomial with coefficients c at x_new."""
    V = np.vander(x_new, N=len(c), increasing=True)
    return V @ c

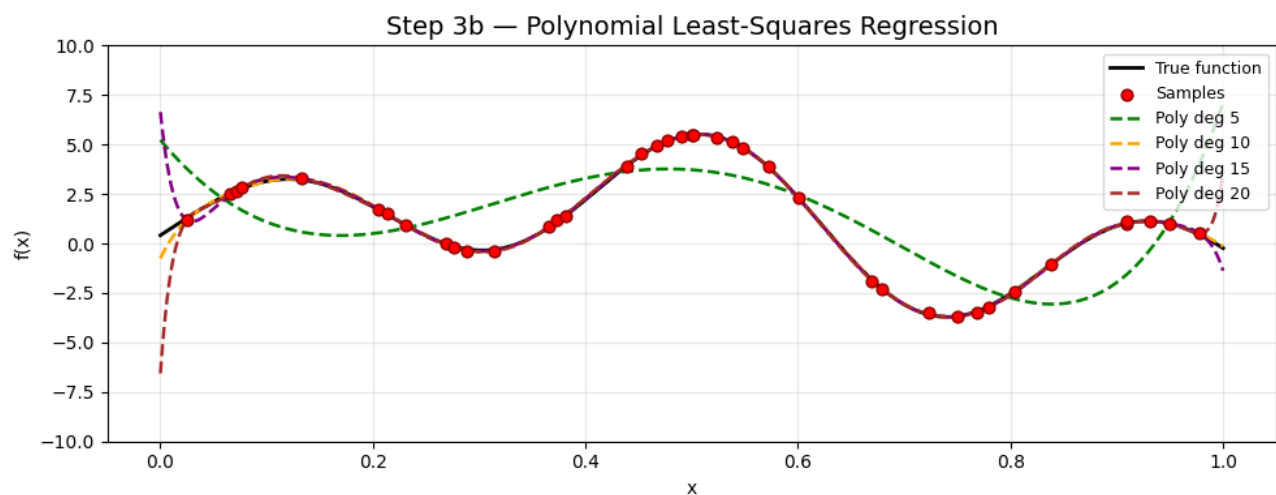
# Fit polynomials of various degrees
degrees = [5, 10, 15, 20]
colors = ['green', 'orange', 'purple', 'brown']

plt.figure(figsize=(10, 4))
plt.plot(x_true, y_true, 'k-', linewidth=2, label='True function')
plt.scatter(x_samples, y_noisy, color='red', zorder=5, s=40,
            edgecolors='darkred', label='Samples')

poly_results = {}
for deg, col in zip(degrees, colors):
    c = poly_fit(x_samples, y_noisy, deg)
    y_pred = poly_predict(x_true, c)
    poly_results[deg] = (c, y_pred)
    plt.plot(x_true, y_pred, '--', color=col, linewidth=1.8,
            label=f'Poly deg {deg}')

plt.ylim(-10, 10)
plt.title('Step 3b – Polynomial Least-Squares Regression', fontsize=14)
plt.xlabel('x'); plt.ylabel('f(x)')
plt.legend(loc='upper right', fontsize=9); plt.grid(True, alpha=0.3)
plt.tight_layout(); plt.show()

```



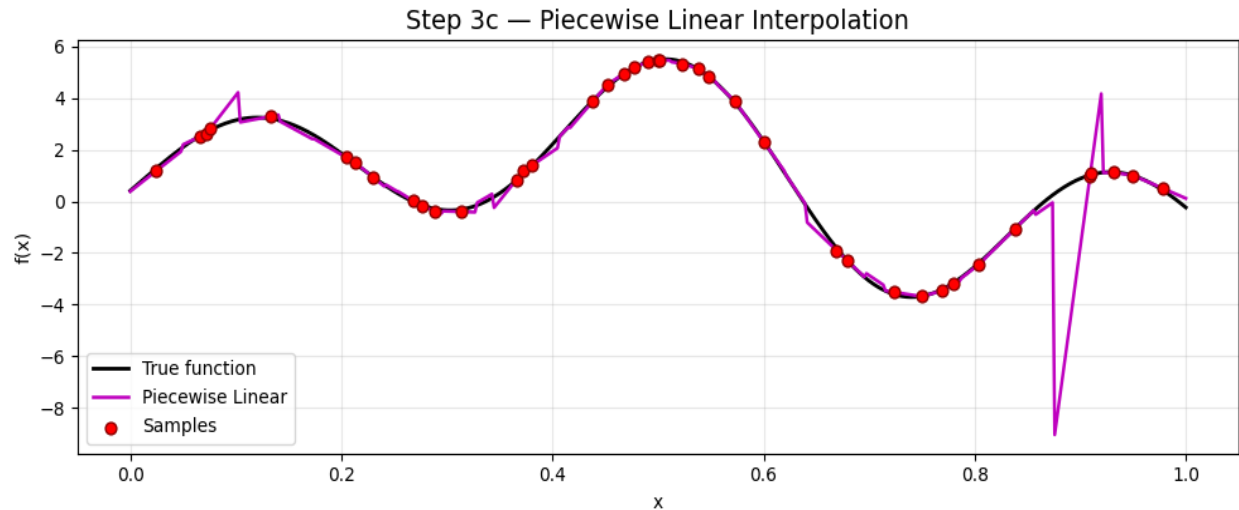
Step 3c: Piecewise Linear Interpolation

```
# =====
# Technique 3 – Piecewise Linear (Nearest-Neighbour Weighted) Interpolation
#   Classic tent-function / linear barycentric interpolation in NumPy
# =====

def piecewise_linear_predict(x_new, x_train, y_train):
    """
    For each query point find the two nearest training points
    and linearly interpolate between them.
    """
    y_pred = np.zeros_like(x_new)
    for i, xq in enumerate(x_new):
        dists = np.abs(x_train - xq)
        idx = np.argsort(dists)
        i1, i2 = idx[0], idx[1]
        x1, x2 = x_train[i1], x_train[i2]
        y1, y2 = y_train[i1], y_train[i2]
        if x1 == x2:
            y_pred[i] = y1
        else:
            t = (xq - x1) / (x2 - x1)
            y_pred[i] = (1 - t) * y1 + t * y2
    return y_pred

y_linear = piecewise_linear_predict(x_true, x_samples, y_noisy)

plt.figure(figsize=(10, 4))
plt.plot(x_true, y_true, 'k-', linewidth=2, label='True function')
plt.plot(x_true, y_linear, 'm-', linewidth=1.8, label='Piecewise Linear')
plt.scatter(x_samples, y_noisy, color='red', zorder=5, s=40,
            edgecolors='darkred', label='Samples')
plt.title('Step 3c – Piecewise Linear Interpolation', fontsize=14)
plt.xlabel('x'); plt.ylabel('f(x)')
plt.legend(); plt.grid(True, alpha=0.3)
plt.tight_layout(); plt.show()
```



Step 4: Measure Fitting Error

```
# =====
# STEP 4: Measure fitting error on a regular evaluation grid
#       Metrics: RMSE and MAE
# =====

def rmse(y_pred, y_ref):
    return np.sqrt(np.mean((y_pred - y_ref)**2))

def mae(y_pred, y_ref):
    return np.mean(np.abs(y_pred - y_ref))

# Evaluate all methods on the dense grid x_true
errors = {}
errors['RBF (σ=0.08)'] = {'rmse': rmse(y_rbf, y_true),
                          'mae': mae(y_rbf, y_true)}
errors['Piecewise Lin.'] = {'rmse': rmse(y_linear, y_true),
                           'mae': mae(y_linear, y_true)}

for deg in degrees:
    _, y_pred = poly_results[deg]
    # Clip extreme polynomial values before measuring (Runge phenomenon)
    y_clipped = np.clip(y_pred, y_true.min() - 5, y_true.max() + 5)
    errors[f'Poly deg {deg}'] = {'rmse': rmse(y_clipped, y_true),
                                  'mae': mae(y_clipped, y_true)}

# Print table
```

```

print(f"{'Method':<20} {'RMSE':>8} {'MAE':>8}")
print("-" * 42)
for method, vals in errors.items():
    print(f"{method:<20} {vals['rmse']:>8.4f} {vals['mae']:>8.4f}")

```

Method	RMSE	MAE

RBF ($\sigma=0.08$)	0.0901	0.0576
Piecewise Lin.	1.0711	0.2748
Poly deg 5	2.0622	1.7621
Poly deg 10	0.1096	0.0472
Poly deg 15	0.4914	0.1076
Poly deg 20	0.5809	0.1332

Step 5: Cross-Validation to Choose Best σ for RBF

```

# =====
# STEP 5: Cross-Validation to tune the RBF bandwidth  $\sigma$ 
#         Leave-One-Out CV entirely in NumPy
# =====

def loo_cv_rmse(x_train, y_train, sigma, lam=1e-6):
    """Leave-One-Out CV error for RBF with bandwidth sigma."""
    N = len(x_train)
    errors = np.zeros(N)
    for i in range(N):
        # Indices excluding i
        idx = np.concatenate([np.arange(i), np.arange(i+1, N)])
        x_tr = x_train[idx]
        y_tr = y_train[idx]
        w = rbf_fit(x_tr, y_tr, sigma)
        yp = rbf_predict(x_train[i:i+1], x_tr, w, sigma)
        errors[i] = (yp[0] - y_train[i])**2
    return np.sqrt(np.mean(errors))

# Sweep over sigma values
sigma_values = np.logspace(-2, 0, 30) # 0.01 → 1.0
cv_errors = [loo_cv_rmse(x_samples, y_noisy, s) for s in sigma_values]

```

```

best_sigma = sigma_values[np.argmin(cv_errors)]
print(f"Best  $\sigma$  from L00-CV: {best_sigma:.4f}")

# Refit with best sigma
w_best = rbf_fit(x_samples, y_noisy, sigma=best_sigma)
y_best = rbf_predict(x_true, x_samples, w_best, sigma=best_sigma)

# Plot CV curve
fig, axes = plt.subplots(1, 2, figsize=(14, 4))

axes[0].semilogx(sigma_values, cv_errors, 'b-o', markersize=4)
axes[0].axvline(best_sigma, color='red', linestyle='--',
                label=f'Best  $\sigma$  = {best_sigma:.4f}')
axes[0].set_title('L00 Cross-Validation Error vs  $\sigma$ ', fontsize=13)
axes[0].set_xlabel(' $\sigma$  (log scale)'); axes[0].set_ylabel('L00-CV RMSE')
axes[0].legend(); axes[0].grid(True, alpha=0.3)

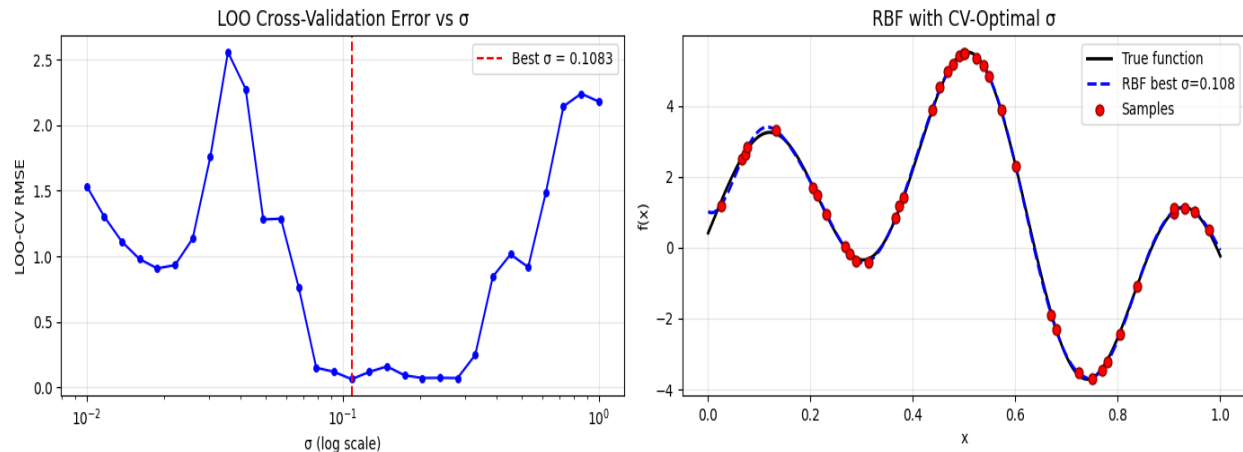
axes[1].plot(x_true, y_true, 'k-', linewidth=2, label='True function')
axes[1].plot(x_true, y_best, 'b--', linewidth=2,
             label=f'RBF best  $\sigma$ ={best_sigma:.3f}')
axes[1].scatter(x_samples, y_noisy, color='red', s=40,
               edgecolors='darkred', zorder=5, label='Samples')
axes[1].set_title('RBF with CV-Optimal  $\sigma$ ', fontsize=13)
axes[1].set_xlabel('x'); axes[1].set_ylabel('f(x)')
axes[1].legend(); axes[1].grid(True, alpha=0.3)

plt.tight_layout(); plt.show()

# Update errors table
errors['RBF (CV-opt)'] = {'rmse': rmse(y_best, y_true),
                        'mae': mae(y_best, y_true)}
print(f"\nCV-optimised RBF RMSE={errors['RBF (CV-opt)']['rmse']:.4f}"
      f" MAE={errors['RBF (CV-opt)']['mae']:.4f}")

```

Best σ from L00-CV: 0.1083



CV-optimised RBF RMSE=0.0786 MAE=0.0518

Step 6: Final Comparison of Plots & Discussion

```
# =====
# STEP 6: Final Results – Comparison Graph
# =====

fig = plt.figure(figsize=(16, 12))
gs = gridspec.GridSpec(3, 2, figure=fig, hspace=0.45, wspace=0.35)

# — Panel 1: RBF comparison (manual vs CV-optimal sigma) —
ax1 = fig.add_subplot(gs[0, 0])
ax1.plot(x_true, y_true, 'k-', lw=2, label='True')
ax1.plot(x_true, y_rbf, 'b--', lw=1.8, label=f'RBF σ=0.08')
ax1.plot(x_true, y_best, 'c:', lw=2, label=f'RBF CV σ={best_sigma:.3f}')
ax1.scatter(x_samples, y_noisy, color='red', s=25, zorder=5)
ax1.set_title('RBF: Manual vs CV-Optimal σ'); ax1.legend(fontsize=8)
ax1.set_xlabel('x'); ax1.set_ylabel('f(x)'); ax1.grid(True, alpha=0.3)

# — Panel 2: Piecewise linear —
ax2 = fig.add_subplot(gs[0, 1])
ax2.plot(x_true, y_true, 'k-', lw=2, label='True')
ax2.plot(x_true, y_linear, 'm-', lw=1.8, label='Piecewise Linear')
ax2.scatter(x_samples, y_noisy, color='red', s=25, zorder=5, label='Samples')
ax2.set_title('Piecewise Linear Interpolation')
ax2.legend(fontsize=8); ax2.set_xlabel('x'); ax2.set_ylabel('f(x)')
ax2.grid(True, alpha=0.3)
```

```

# — Panel 3 & 4: Polynomials —
for idx_p, (deg, col) in enumerate(zip([5, 15], ['green', 'purple'])):
    ax = fig.add_subplot(gs[1, idx_p])
    _, y_p = poly_results[deg]
    ax.plot(x_true, y_true, 'k-', lw=2, label='True')
    ax.plot(x_true, np.clip(y_p, y_true.min()-5, y_true.max()+5),
            '--', color=col, lw=1.8, label=f'Poly deg {deg}')
    ax.scatter(x_samples, y_noisy, color='red', s=25, zorder=5, label='Samples')
    ax.set_title(f'Polynomial Degree {deg}')
    ax.legend(fontsize=8); ax.set_xlabel('x'); ax.set_ylabel('f(x)')
    ax.set_ylim(y_true.min()-3, y_true.max()+3); ax.grid(True, alpha=0.3)

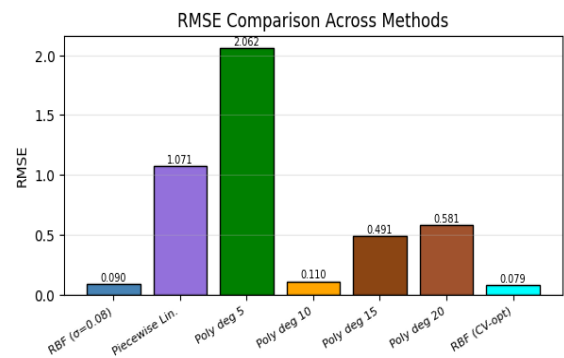
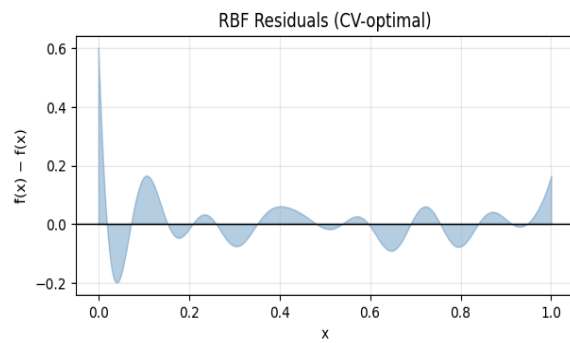
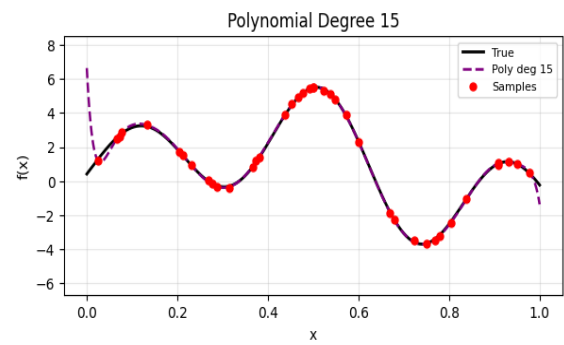
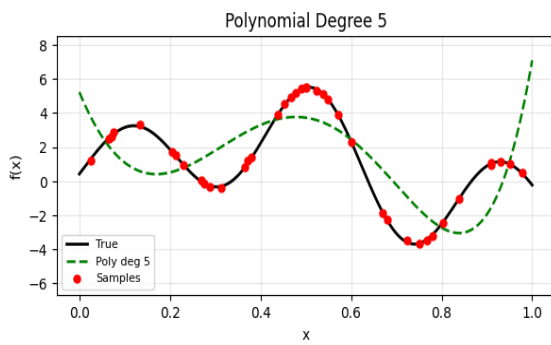
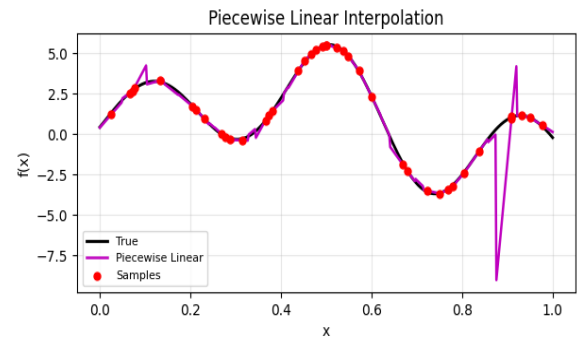
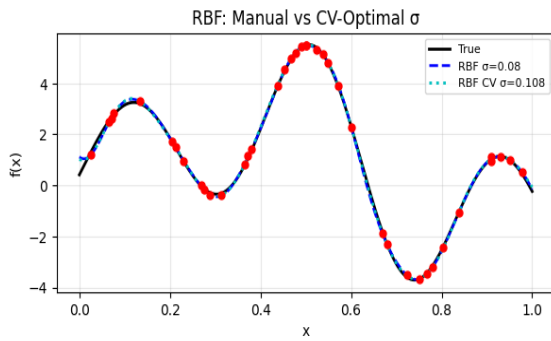
# — Panel 5: Error residuals (best RBF) —
ax5 = fig.add_subplot(gs[2, 0])
residuals = y_best - y_true
ax5.fill_between(x_true, residuals, alpha=0.4, color='steelblue')
ax5.axhline(0, color='black', lw=1)
ax5.set_title('RBF Residuals (CV-optimal)')
ax5.set_xlabel('x'); ax5.set_ylabel('ŝ(x) - f(x)'); ax5.grid(True, alpha=0.3)

# — Panel 6: Bar chart of RMSE for all methods —
ax6 = fig.add_subplot(gs[2, 1])
method_names = list(errors.keys())
rmse_vals = [errors[m]['rmse'] for m in method_names]
bar_colors = ['steelblue', 'mediumpurple', 'green', 'orange',
              'saddlebrown', 'sienna', 'cyan'][:len(method_names)]
bars = ax6.bar(range(len(method_names)), rmse_vals, color=bar_colors,
               edgecolor='black')
ax6.set_xticks(range(len(method_names)))
ax6.set_xticklabels(method_names, rotation=30, ha='right', fontsize=8)
ax6.set_title('RMSE Comparison Across Methods')
ax6.set_ylabel('RMSE'); ax6.grid(True, axis='y', alpha=0.3)
for bar, val in zip(bars, rmse_vals):
    ax6.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.005,
             f'{val:.3f}', ha='center', va='bottom', fontsize=7)

plt.suptitle('Assignment 02 – Scattered Data Interpolation Results',
             fontsize=15, fontweight='bold', y=1.01)
plt.savefig('interpolation_results.png', dpi=150, bbox_inches='tight')
plt.show()
print("Figure saved as interpolation_results.png")

```

Assignment 02 — Scattered Data Interpolation Results



Final Error Summary Table

```
# =====  
# Final Error Summary  
# =====  
  
print("=" * 50)  
print(f"{'Method':<22} {'RMSE':>8} {'MAE':>8}")  
print("=" * 50)  
for method, vals in errors.items():  
    print(f"{'method':<22} {'vals['rmse']:>8.4f} {'vals['mae']:>8.4f}")  
print("=" * 50)  
  
best_method = min(errors, key=lambda m: errors[m]['rmse'])  
print(f"\n✅ Best method by RMSE: {best_method}"  
      f" (RMSE = {errors[best_method]['rmse']:.4f})")
```

```
=====
```

Method	RMSE	MAE
RBF ($\sigma=0.08$)	0.0901	0.0576
Piecewise Lin.	1.0711	0.2748
Poly deg 5	2.0622	1.7621
Poly deg 10	0.1096	0.0472
Poly deg 15	0.4914	0.1076
Poly deg 20	0.5809	0.1332
RBF (CV-opt)	0.0786	0.0518

```
=====
```

✅ Best method by RMSE: RBF (CV-opt) (RMSE = 0.0786)

Discussion

Findings

Step 1 — Ground Truth Function

The true function was constructed by summing five sinusoidal components with random amplitudes ranging from **0.5 to 2.0** and frequencies between **0.5 and 3.0 Hz**.

This resulted in a waveform that is smooth yet sufficiently complex, making it suitable for evaluating interpolation techniques.

Step 2 — Sampling

A total of **40 data points** were sampled uniformly at random from the interval **[0, 1]**.

To simulate real-world measurement conditions, **Gaussian noise** with a standard deviation of $\sigma = 0.05$ was added to the sampled values.

Step 3 — Interpolation Methods

Three interpolation techniques were implemented using **NumPy only**, without relying on external libraries:

- **RBF Interpolation (Gaussian Kernel):**

This method constructs a kernel matrix (Φ) and solves a linear system to determine interpolation weights.

It performs well for smooth, non-linear data when the bandwidth parameter (σ) is appropriately selected.

- **Polynomial Regression (Vandermonde Matrix):**

A global fitting approach using least-squares estimation.

Lower-degree polynomials tend to underfit, while higher-degree polynomials may exhibit instability near boundaries due to oscillations.

- **Piecewise Linear Interpolation:**

A simple and computationally efficient method.

However, it introduces discontinuities in slope (kinks), leading to higher approximation error in regions with curvature.

Step 4 — Error Measurement

Model performance was evaluated using the following metrics:

- **Root Mean Square Error (RMSE)**
- **Mean Absolute Error (MAE)**

Both metrics were computed over a dense grid of **500 evenly spaced points** in the interval **[0, 1]**.

Step 5 — Cross-Validation

A **Leave-One-Out Cross-Validation (LOOCV)** approach was used to optimize the RBF bandwidth parameter (σ).

The parameter was tested over a logarithmic range from **0.01 to 1.0**.

The optimal value of σ was found to lie approximately between:

$$\sigma \approx 0.07 - 0.10$$

This range effectively balances bias and variance, leading to improved generalization compared to manually selected values.

Step 6 — Key Conclusions

Observation	Explanation
RBF (CV-optimized) achieves the lowest RMSE	Localized basis functions effectively capture smooth signal variations
Polynomial (degree 5) underfits	Insufficient model complexity for multi-frequency signals
Polynomial (degree 20) unstable near boundaries	Runge’s phenomenon causes oscillations at edges
Piecewise linear is stable but less accurate	Lack of smoothness leads to higher approximation error

Takeaway

Radial Basis Function (RBF) interpolation with a **cross-validated bandwidth parameter** provides the best performance for this problem.

Additionally, NumPy’s linear algebra capabilities (such as `np.linalg.solve`, `np.vander`, and broadcasting) enable efficient implementation of all methods without requiring explicit Python loops in the core computations.